

CRYPTOGRAPHY AND DATA SECURITY

PROJECT PROPOSAL

RSN (Root-based Substitution and Number Theory) Cipher Algorithm

Problem Statement:

In modern communication scenarios, protecting sensitive information from unauthorized access is a core essential requirement. The six classic ciphers, namely Caesar, Vigenère, Playfair, Hill, Rail Fence, and Columnar Transposition, execute encryption through substitution, shifting, matrix transformation, and permutation logic. However, when used alone, they have notable security flaws, and can be compromised by frequency analysis, pattern recognition, and known-plaintext attacks.

This project will develop a novel symmetric encryption algorithm that integrates classic ciphers with number theory concepts such as prime numbers and modular arithmetic, alongside a supporting secure key sharing process to ensure that both the sender and receiver can securely acquire the same encryption key.

Proposed Algorithm:

The proposed algorithm is named the RSN Algorithm.

RSN is a symmetric block cipher encryption algorithm where the plain text is converted to a number, packed into an integer block of a very large size, and subjected to multiple rounds of transformations. RSN makes use of a large Mersenne Prime number such as:

$$P = 2^{61} - 1$$

$$P = 2305843009213693951$$

Instead of encrypting letters directly using modulo 26, the RSN Algorithm first maps characters into a 29-symbol character set:

A-Z, SPACE, . and ?

Characters are represented by numbers ranging from 0 to 28. Numbers obtained are then packed using base 29 and encrypted within the large prime number modulo P.

The RSN algorithm utilizes the principles of Feistel network, as used in DES. The input block is first divided into two halves, the left and right halves. At each round, the right half is shuffled by means of a round function and then combined with the left half.

Techniques Used:

The RSN Algorithm uses ideas from both studied classical cryptography and number theorems.

Technique	Concept	Purpose
Character mapping	Classical substitution ciphers	Converts plaintext into numbers
Base-29 packing	Modular representation	Groups many characters into one large number
Large prime modulus	Prime numbers and modular arithmetic	Creates a large encryption space
Round key addition	Caesar and Vigenere ciphers	Adds key-based confusion
Primitive root exponentiation	Primitive roots and Euler's Totient Function	Adds nonlinear substitution
Matrix multiplication	Hill cipher	Provides diffusion
Key-based permutation	Rail Fence, Route, Columnar Transposition	Rearranges positions of values
Feistel structure	DES	Makes encryption and decryption reversible
Modular inverse	GCD and relative primes	Used in key sharing
Three-pass key sharing	Number theory	Allows secure sharing of the symmetric key

Steps of the RSN Encryption Algorithm:

Step 1: Plaintext Preparation

The plaintext is converted to uppercase, unsupported characters are removed or replaced, and padding is added if needed.

Step 2: Character Conversion

Each character is converted into a number using the defined 29-symbol character set.

Step 3: Block Formation

The numeric characters are packed into large number blocks and split into left and right halves.

Step 4: Round Key Generation

The secret key is used to generate separate round keys, matrices, and permutations for each round.

Step 5: Round Processing

Each round applies key mixing, primitive-root substitution, matrix diffusion, and key-based permutation to the right half.

Step 6: Feistel Combination

The processed right half is combined with the left half using modular arithmetic, and the halves are swapped.

Step 7: Repetition

The round process is repeated for 16 rounds to increase confusion and diffusion.

Step 8: Ciphertext Output

After the final round, both halves are combined and converted into ciphertext in base64 form.

Key Sharing Process:

Step 1: Public Setup

Alice and Bob agree on a large public prime number to be used in the key-sharing process.

Step 2: Session Key Selection

Alice selects a random session key that will later be used as the RSN Seed value.

Step 3: Alice's Lock

Alice applies her private mathematical lock to the session key and sends the locked value to Bob.

Step 4: Bob's Lock

Bob applies his private mathematical lock to Alice's locked value and sends the double-locked value back to Alice.

Step 5: Alice Unlocks

Alice removes only her own lock and sends the remaining Bob locked value back to Bob.

Step 6: Bob Unlocks

Bob removes his own lock and recovers the original session key.

Step 7: Shared Key Usage

Both Alice and Bob use the recovered session key to generate the same RSN round keys for encryption and decryption.

Security Justification:

The RSN encryption algorithm is meant to be more robust compared to basic classical cryptography owing to the use of more than one method in its structure. Such methods include substitution, transposition, matrix diffusion, modular arithmetic, and many rounds of encryption.

Use of a large prime modulus results in an increase in size of the mathematical field inside the cryptosystem. Confusion will be brought about by primitive-root substitution, while diffusion will come from matrix multiplication and permutation. The Feistel structure ensures that encryption and decryption remain possible using the same symmetric key.

The key distribution process enhances the design since the session key is never transmitted. When the attacker intercepts the transmitted information, they can only obtain locked variants of the session key.